Часть 1. Основы Python • Тема 1.5 • Занятие 1 из 2

# 1.5. Циклы

**К концу занятия ты сможешь:**

- повторять действия, пока условие верно
- считать шаги и накапливать сумму в цикле
- повторять действие нужное число раз и перебирать строку по буквам
- выходить из цикла досрочно и пропускать шаги

# Разминка

**1.** Что выведет этот код? (тема 1.3)

```
main.py
score = 10
score += 5
score *= 2
print(score)
```

**2.** Чем elif отличается от второго if, написанного ниже первого? (тема 1.4)

**3.** Как одним условием проверить, что возраст от 14 до 17 включительно? (тема 1.4)

# Задача, которую пока нечем решить

Сейф открывается только кодом **1234**. Программа должна спрашивать код снова и снова, пока его не введут верно.

Сколько раз написать input()? Один угадает с первой попытки, другой — с десятой. Заранее этого не знает никто.

**Сегодня ты напишешь такую программу за 5 строк.**

Вывод в консоли

```
Введи код сейфа: 0000   <- ввёл пользователь
Неверно
Введи код сейфа: 1111   <- ввёл пользователь
Неверно
Введи код сейфа: 1234   <- ввёл пользователь
Сейф открыт
```

Стрелка — пометка для тебя, в программе её нет

# План занятия

УРОК 1

## Цикл while

Повторять, пока условие верно. Счётчик и накопитель. Как остановить зависшую программу.

Практика: «Копилка на мечту»

УРОК 2

## Цикл for и range

Повторить нужное число раз. Начало, конец и шаг. Перебор строки по буквам.

Практика: «Фитнес-план»

УРОК 3

## break и continue

Выйти из цикла досрочно. Пропустить шаг.

Практика: «Плейлист на тренировку». Мини-проект «Шагомер недели»

# Вспомним: условия и составное присваивание

**ТЕМА 1.3**

lives -= 1 — то же, что lives = lives - 1

coins += 10 — то же, что coins = coins + 10

% — остаток от деления: 8 % 4 даёт 0, 9 % 4 даёт 1

**ТЕМА 1.4**

Условие даёт True или False: == != < <= > >=

После if — двоеточие, тело с отступом 4 пробела

= присваивает, == сравнивает

Блок 1 из 3 • Урок 1

# Цикл while

**В этом блоке ты научишься:**

- повторять действия, пока условие верно
- считать шаги и накапливать сумму
- останавливать зависшую программу и находить причину

# Что такое цикл и зачем он нужен

**Цикл** — команда «повторяй». Программа выполняет одни и те же строки много раз.

Как в жизни: «пока в стакане есть вода — пей». Ты не знаешь заранее, сколько будет глотков.

Без цикла пять раундов игры — пять одинаковых строк. А если раундов 100? А если их число вводит игрок?

main.py — без цикла

```
print("Раунд 1")
print("Раунд 2")
print("Раунд 3")
print("Раунд 4")
print("Раунд 5")
```

Вывод в консоли

```
Раунд 1
Раунд 2
Раунд 3
Раунд 4
Раунд 5
```

# Как записывается while

```
шаблон
while условие:
    действие 1
    действие 2
следующая команда
```

Python проверяет условие. **True** — выполняет тело и снова проверяет. **False** — переходит к следующей команде.

while — «пока»

**условие** — такое же, как в if (тема 1.4)

**двоеточие** после условия обязательно

**тело цикла** — строки с отступом 4 пробела, они повторяются

строка **без отступа** — уже после цикла, выполнится один раз

**while повторяет тело, пока условие верно.**

# Пример: обратный отсчёт

main.py

```
count = 5  # с какого числа начинаем
while count > 0:  # пока больше нуля
    print(count)
    count -= 1  # уменьшаем на 1
print("Пуск")
```

Вывод в консоли

```
5
4
3
2
1
Пуск
```

**Строка 2** — пока count больше нуля, повторяй

**Строка 4** — меняет count, иначе условие не станет False

**Строка 5** — без отступа: выполнится один раз после цикла

# Трассировка: что происходит на каждом шаге

| Шаг | count | count > 0 | Что произошло |
|---|---|---|---|
| 1 | 5 | True | печать 5, count = 4 |
| 2 | 4 | True | печать 4, count = 3 |
| 3 | 3 | True | печать 3, count = 2 |
| 4 | 2 | True | печать 2, count = 1 |
| 5 | 1 | True | печать 1, count = 0 |
| 6 | 0 | False | цикл закончен, печать «Пуск» |

**Тело выполнилось 5 раз, а условие проверялось 6 раз: последняя проверка и остановила цикл.**

# Счётчик: считаем шаги

**Счётчик** — переменная, которая считает шаги. Как кликер у охранника на входе: каждый человек — плюс один.

main.py

```
round_number = 1  # счётчик
while round_number <= 5:
    print(f"Раунд {round_number}")
    round_number += 1  # следующий раунд
print("Игра окончена")
```

Вывод в консоли

```
Раунд 1
Раунд 2
Раунд 3
Раунд 4
Раунд 5
Игра окончена
```

**Переменная из условия должна меняться в теле цикла.**

# Накопитель: собираем сумму

**Накопитель** — переменная, куда на каждом шаге что-то добавляют. Как копилка: сначала пусто, потом всё больше.

main.py

```
savings = 0  # накопитель: сначала пусто
day = 1
while day <= 4:
    savings += 150  # кладём 150 рублей
    print(f"День {day}: в копилке {savings} руб.")
    day += 1
print(f"Итого: {savings} руб.")
```

**Счётчик прибавляет 1, накопитель — нужное значение.**

Вывод в консоли

```
День 1: в копилке 150 руб.
День 2: в копилке 300 руб.
День 3: в копилке 450 руб.
День 4: в копилке 600 руб.
Итого: 600 руб.
```

Здесь два «хранилища»: day — счётчик, savings — накопитель.

# Ловушка: бесконечный цикл и как его остановить

main.py

```
count = 5
while count > 0:
    print(count)
    # забыли count -= 1
print("Пуск")
```

Вывод в консоли (без конца)

```
5
5
5
5
```

count всегда 5, условие всегда True — **бесконечный цикл**. Программа не закончится сама.

**Остановить:** в терминале VS Code — **Ctrl+C**; в Thonny — красная кнопка **«Стоп»**.

После Ctrl+C появится traceback — сообщение об ошибке. Последняя строка KeyboardInterrupt значит: программу остановил ты.

Место для скриншота: терминал VS Code после Ctrl+C и кнопка «Стоп» в Thonny

# Ловушка: input() до цикла

## Зависает

main.py

```
code = input("Введи код сейфа: ")
while code != "1234":
    print("Неверно")
print("Сейф открыт")
```

Вывод в консоли (без конца)

```
Введи код сейфа: 0000   <- ввёл пользователь
Неверно
Неверно
Неверно
```

## Работает

main.py

```
code = input("Введи код сейфа: ")
while code != "1234":
    print("Неверно")
    code = input("Введи код сейфа: ")  # новый ввод
print("Сейф открыт")
```

Вывод в консоли

```
Введи код сейфа: 0000   <- ввёл пользователь
Неверно
Введи код сейфа: 1111   <- ввёл пользователь
Неверно
Введи код сейфа: 1234   <- ввёл пользователь
Сейф открыт
```

**Слева code не меняется — ввод нужен внутри цикла. Стрелка — пометка для тебя.**

# Живой кодинг: «Копилка до цели»

Сколько недель копить на наушники?

1. Спросить цену наушников и сколько откладываешь в неделю
2. Создать накопитель savings = 0 и счётчик weeks = 0
3. Пока savings меньше цены: добавить взнос, weeks += 1, вывести неделю
4. После цикла вывести, через сколько недель хватит

Что получится

```
Сколько стоят наушники? 3500   <- ввёл пользователь
Сколько откладываешь в неделю? 800   <- ввёл пользователь
Неделя 1: 800 руб.
Неделя 2: 1600 руб.
Неделя 3: 2400 руб.
Неделя 4: 3200 руб.
Неделя 5: 4000 руб.
Хватит через 5 нед.
```

Стрелка — пометка для тебя, в программе её нет

# Вопросы по блоку 1

**1.** Что выведет этот код? Сколько раз выполнится тело цикла?

main.py

```
n = 1
while n < 20:
    n *= 3
    print(n)
```

**2.** Почему программа «Сейф», где input() стоит только до цикла, никогда не заканчивается?

**3.** Что будет, если в обратном отсчёте убрать строку count -= 1? Как остановить такую программу?

# Вопросы по блоку 1

**4.** Чем счётчик отличается от накопителя? Приведи пример каждого из жизни или из игры.

**5.** Обратный отсчёт начинается с 5. Сколько раз выполнится тело цикла и сколько раз Python проверит условие? Как это видно по таблице трассировки?

# Задание: «Копилка на мечту»

1. Программа спрашивает, сколько стоит мечта (самокат, игра, билет на концерт).

2. Затем снова и снова спрашивает «Сколько добавляешь?», пока накоплено меньше цели.

3. В конце пишет, сколько накоплено и сколько было взносов.

**Подсказка:** нужны накопитель для суммы и счётчик для взносов. input() стоит внутри цикла.

★ **Успел раньше?** После каждого взноса пиши, сколько ещё осталось до цели.

Застрял — 15 минут ищи сам, потом спроси: что делал, что ожидал, что получил.

# Что должно получиться

Вывод в консоли

```
Сколько стоит мечта? 5000   <- ввёл пользователь
Сколько добавляешь? 1500   <- ввёл пользователь
Сколько добавляешь? 2000   <- ввёл пользователь
Сколько добавляешь? 1800   <- ввёл пользователь
Цель достигнута. Накоплено: 5300 руб.
Число взносов: 3
```

Стрелка — пометка для тебя, в программе её нет

## Проверь себя

- Если первый взнос сразу больше цели, вопрос задаётся один раз и взносов — 1
- Программа заканчивается сама, без Ctrl+C
- Ты можешь объяснить каждую строку своего кода

# Загадка Коллатца

Возьми любое число больше нуля. Если оно чётное — раздели пополам. Если нечётное — умножь на 3 и прибавь 1. Повторяй.

Математики верят, что так из любого числа рано или поздно получится 1, но доказать это пока не смог никто.

Напиши программу: она спрашивает число, печатает весь путь до 1 в одну строку и считает шаги.

Пример результата

```
Число: 6   <- ввёл пользователь
3 10 5 16 8 4 2 1
Шагов до 1: 8
```

Попробуй 27. Сколько будет шагов?

Блок 2 из 3 • Урок 2

# Цикл for и range

**В этом блоке ты научишься:**

- повторять действие нужное число раз
- задавать начало, конец и шаг счёта
- перебирать строку по буквам

# Зачем нужен for

**for** — цикл «для каждого». Он сам берёт значения по очереди, как учитель, который вызывает учеников по журналу: первый, второй, третий.

### while: счётчик в трёх местах

main.py

```
round_number = 1
while round_number <= 3:
    print(f"Раунд {round_number}")
    round_number += 1
```

### for: счётчик в одной строке

main.py

```
for round_number in range(1, 4):
    print(f"Раунд {round_number}")
```

### Вывод у обеих

Вывод в консоли

```
Раунд 1
Раунд 2
Раунд 3
```

**Знаешь заранее, сколько повторов, — бери for. Не знаешь — while.**

# Как записывается for

шаблон

```
for переменная in range(n):
    действие
следующая команда
```

**range(n)** — «диапазон»: набор чисел от 0 до n – 1. Сам он ничего не печатает, он только выдаёт числа циклу.

for — «для каждого»

**переменная** — получает значения по очереди, создавать её заранее не нужно

in — «из»

**двоеточие и отступ** — как в while и if

**for сам меняет переменную: += 1 писать не надо.**

# Пример: range(5) считает с нуля

main.py

```
for i in range(5):
    print(f"i = {i}")
```

Вывод в консоли

```
i = 0
i = 1
i = 2
i = 3
i = 4
```

| Шаг | i |
|---|---|
| 1 | 0 |
| 2 | 1 |
| 3 | 2 |
| 4 | 3 |
| 5 | 4 |

**range(5) — пять чисел: 0, 1, 2, 3, 4. Пятёрки среди них нет.**

# range(start, stop): своё начало

```
main.py
for level in range(1, 6):
    print(f"Уровень {level} пройден")
```

**start** — с какого числа начать, оно **входит**

**stop** — на каком остановиться, оно **не входит**

range(1, 6) → 1, 2, 3, 4, 5

**stop — это «стоп-линия»: до неё доходим, но не наступаем.**

```
Вывод в консоли
Уровень 1 пройден
Уровень 2 пройден
Уровень 3 пройден
Уровень 4 пройден
Уровень 5 пройден
```

# Ловушка: range(1, 10) не доходит до 10

Задача: вывести числа от 1 до 10 в одну строку (end=" " — из темы 1.2).

**Было: нет десятки**

main.py

```
for number in range(1, 10):
    print(number, end=" ")
```

Вывод в консоли

```
1 2 3 4 5 6 7 8 9
```

**Стало: до 10 включительно**

main.py

```
for number in range(1, 11):
    print(number, end=" ")
```

Вывод в консоли

```
1 2 3 4 5 6 7 8 9 10
```

**Нужно до N включительно — пиши range(1, N + 1).**

# range(start, stop, step): свой шаг

main.py

```
for seconds in range(0, 21, 5):
    print(seconds, end=" ")
print()
for count in range(10, 0, -2):
    print(count, end=" ")
```

Вывод в консоли

```
0 5 10 15 20
10 8 6 4 2
```

**step** — на сколько меняется число на каждом шаге (по умолчанию 1)

**Отрицательный шаг** — счёт назад. Тогда start больше stop

print() без аргументов — просто переход на новую строку

**stop не входит и здесь: 21 не печатается, 0 тоже.**

# Живой кодинг: таблица умножения

Шпаргалка к контрольной на любое число.

1. Спросить, на какое число таблица
2. Повторить для i от 1 до 10 включительно
3. На каждом шаге вывести строку «число x i = результат»
4. Запустить и проверить последнюю строку: есть ли 10?

Что получится

```
Таблица умножения на: 7   <- ввёл пользователь
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Стрелка — пометка для тебя, в программе её нет

# Перебор строки по символам

Строка — это цепочка символов, как бусины на нитке. for может пройти по ней и выдать символы по одному.

```
main.py
for letter in "Python":
    print(letter)
```

**После in можно поставить строку: for выдаст её символы слева направо.**

```
Вывод
P
y
t
h
o
n
```

| Шаг | letter |
|---|---|
| 1 | P |
| 2 | y |
| 3 | t |
| 4 | h |
| 5 | o |
| 6 | n |

# Счётчик внутри for: ищем букву

main.py

```
phrase = "голосуем за новый трек"
count = 0
for letter in phrase:
    if letter == "о":
        count += 1
print(f"Буква о встретилась: {count}")
```

Вывод в консоли

```
Буква о встретилась: 3
```

**Строка 2** — счётчик создаём до цикла

**Строки 4–5** — if внутри цикла: считаем только подходящие буквы

**Строка 6** — без отступа: печать один раз, после цикла

**Счётчик и накопитель из блока 1 работают и в for.**

# Вопросы по блоку 2

**1.** Что выведет этот код?

main.py

```
for i in range(2, 10, 3):
    print(i, end=" ")
```

**2.** Почему range(1, 10) не печатает 10? Как исправить, не меняя первое число?

# Вопросы по блоку 2

**3.** Что выведет этот код? Почему?

main.py

```
for i in range(5, 1):
    print(i, end=" ")
```

**4.** Когда удобнее while, а когда for? Приведи по одному примеру программы для каждого.

# Задание: «Фитнес-план»

1. Программа спрашивает, сколько недель длится план и сколько приседаний делать в первую неделю.

2. Каждую следующую неделю приседаний на 5 больше. Для каждой недели выводится строка «Неделя N: … приседаний».

3. В конце — сколько приседаний всего за весь план.

**Подсказка:** недели — for week in range(1, weeks + 1). Итог — накопитель из блока 1.

★ **Успел раньше?** Каждая 4-я неделя — отдых: пиши «Неделя 4: отдых» и не прибавляй её к итогу (подойдут % и if).

Последняя неделя пропала? Проверь stop в range.

# Что должно получиться

Вывод в консоли

```
Сколько недель? 4   <- ввёл пользователь
Приседаний в первую неделю: 20   <- ввёл пользователь
Неделя 1: 20 приседаний
Неделя 2: 25 приседаний
Неделя 3: 30 приседаний
Неделя 4: 35 приседаний
Всего за план: 110
```

Стрелка — пометка для тебя, в программе её нет

## Проверь себя

- Строк с неделями ровно столько, сколько ввёл
- Первая неделя — то число, которое ввёл
- При 1 неделе итог равен стартовому числу

# «Счастливый билет»

Билет в автобусе счастливый, если сумма первых трёх цифр равна сумме последних трёх.

Программа спрашивает номер из 6 цифр, считает обе суммы и говорит, счастливый ли билет.

Условие: перебирай строку через for. Какая сейчас по счёту цифра — придумай сам.

Пример результата

```
Номер билета: 123402   <- ввёл пользователь
Слева: 6, справа: 6
Счастливый билет
```

Блок 3 из 3 • Урок 3

# break и continue

**В этом блоке ты научишься:**

- выходить из цикла досрочно
- пропускать шаг цикла и переходить к следующему

# break: выйти из цикла досрочно

**break** — «стоп, выходим». Цикл заканчивается сразу, даже если шаги ещё остались.

Ищешь ключи по карманам: нашёл во втором — третий уже не проверяешь.

Зачем: у пароля 3 попытки, но если ввели верно с первой, спрашивать ещё два раза глупо.

шаблон

```
for ... in ...:
    if условие:
        break
    действие
первая строка после цикла
```

**break почти всегда стоит внутри if. После break Python идёт к первой строке после цикла.**

# Пример: вход с тремя попытками

main.py

```python
for attempt in range(1, 4):
    password = input(f"Попытка {attempt}. Пароль: ")
    if password == "dragon":
        print("Вход выполнен")
        break  # выходим из цикла
    print("Неверный пароль")
print("Конец программы")
```

Вывод в консоли

```
Попытка 1. Пароль: qwerty   <- ввёл пользователь
Неверный пароль
Попытка 2. Пароль: dragon   <- ввёл пользователь
Вход выполнен
Конец программы
```

Стрелка — пометка для тебя, в программе её нет

**Строка 5** — break: попытка 3 уже не спрашивается

**Строка 6** — выполняется, только если break не сработал

# continue: пропустить шаг

**continue** — «этот шаг пропускаем». Остаток тела не выполняется, цикл сразу идёт к следующему шагу.

Как кнопка «следующий трек»: эта песня не нравится — плеер не выключается, а переходит к следующей.

Пример: ник для игры без подчёркиваний.

main.py

```
for letter in "shadow_fox":
    if letter == "_":
        continue  # пропускаем этот шаг
    print(letter, end="")
```

Вывод в консоли

```
shadowfox
```

**continue пропускает один шаг, а цикл продолжается.**

# break и continue на одном коде

**break: выключает плеер**

main.py

```
for track in range(1, 7):
    if track == 4:
        break
    print(f"Играет трек {track}")
print("Плеер выключен")
```

Вывод в консоли

```
Играет трек 1
Играет трек 2
Играет трек 3
Плеер выключен
```

**continue: пропускает трек 4**

main.py

```
for track in range(1, 7):
    if track == 4:
        continue
    print(f"Играет трек {track}")
print("Плеер выключен")
```

Вывод в консоли

```
Играет трек 1
Играет трек 2
Играет трек 3
Играет трек 5
Играет трек 6
Плеер выключен
```

# Ловушка: continue в while

## Зависает

main.py

```
day = 0
while day < 7:
    if day == 3:
        continue
    print(f"День {day}")
    day += 1
```

Вывод в консоли

```
День 0
День 1
День 2
```

Дальше тишина, но программа не закончилась. Ctrl+C или «Стоп».

## Работает

main.py

```
day = 0
while day < 7:
    day += 1  # меняем счётчик в начале
    if day == 3:
        continue
    print(f"День {day}")
```

Вывод в консоли

```
День 1
День 2
День 4
День 5
День 6
День 7
```

# Вопросы по блоку 3

**1.** Чем break отличается от continue? Объясни на примере плеера.

**2.** В программе «Вход с тремя попытками» пользователь три раза ввёл неверный пароль. Что напечатается и какой будет последняя строка?

**3.** Почему while с continue из ловушки зависает? Что нужно переставить, чтобы он заработал?

# Задание: «Плейлист на тренировку»

1. Программа по очереди спрашивает длительность 5 песен в минутах.

2. Если ввели 0 — пишет «Пропускаем», и эта песня не считается.

3. Как только сумма стала 30 минут или больше — пишет «Тренировка набрана» и больше не спрашивает.

4. В конце — сколько всего минут.

**Подсказка:** 0 — это continue, «набрано» — это break. Оба внутри if.

★ **Успел раньше?** В конце напиши, сколько песен прозвучало (пропущенные не считаются).

# Что должно получиться

Вывод в консоли

```
Песня 1, минут: 8   <- ввёл пользователь
Песня 2, минут: 0   <- ввёл пользователь
Пропускаем
Песня 3, минут: 12   <- ввёл пользователь
Песня 4, минут: 11   <- ввёл пользователь
Тренировка набрана
Всего минут: 31
```

Стрелка — пометка для тебя, в программе её нет

### Проверь себя

- Песню 5 программа не спросила — сработал break
- Ноль не прибавился к сумме
- Если все песни короткие, вопросов ровно 5

# Где первая цифра в пароле?

Программа спрашивает пароль и находит первую цифру в нём: какая это цифра и на каком она месте (считая с 1).

Если цифр нет — пишет «Цифр нет».

Как понять, что символ — цифра, вспомни из темы 1.2. Как после цикла узнать, нашлась ли цифра, — придумай сам.

Пример результата

```
Пароль: dragon7x   <- ввёл пользователь
Первая цифра 7 на позиции 7
```

# Вопросы по занятию

**1.** Что будет, если в этом коде поменять range(1, 8) на range(8)?

main.py

```
for i in range(1, 8):
    print(i, end=" ")
```

**2.** Программу «Сейф» можно написать через for? А «Таблицу умножения» через while? Что удобнее и почему?

**3.** Где в сегодняшних программах условие из темы 1.4 стояло в самом while, а где — в if внутри цикла?

# «Шагомер недели»

Свой шагомер, как в телефоне: ставишь цель на неделю, каждый день вводишь шаги, программа считает итог и сама замечает, когда цель выполнена.

В нём работают все три блока: while, for и накопитель, break и continue.

Это пригодится в Проекте 5 «Личный трекер»: там тоже нужно много раз принимать записи и копить итог.

Пример работы

```
Как тебя зовут? Максим   <- ввёл пользователь
Цель на неделю, шагов: -5   <- ввёл пользователь
Цель должна быть больше нуля
Цель на неделю, шагов: 30000   <- ввёл пользователь
День 1, шагов: 8000   <- ввёл пользователь
Итого: 8000
День 2, шагов: -1   <- ввёл пользователь
День пропущен
День 3, шагов: 12000   <- ввёл пользователь
Итого: 20000
День 4, шагов: 11000   <- ввёл пользователь
Итого: 31000
Цель выполнена
За неделю: 31000 шагов
Отчёт для: М а к с и м
```

Стрелка — пометка для тебя, в программе её нет

# Что должен уметь шагомер

**1**

Спрашивает имя и цель на неделю. Если цель 0 или меньше — просит ввести снова.

**2**

7 дней подряд спрашивает, сколько шагов пройдено, и показывает итог.

**3**

–1 значит «день пропущен». Когда цель выполнена — поздравляет и больше не спрашивает.

**4**

В конце пишет шаги за неделю и имя по буквам через пробел.

# План по шагам

| Шаг | Что сделать | Тема | Запусти — должно быть |
|---|---|---|---|
| 1 | Спросить имя и цель; пока цель 0 или меньше — переспрашивать | 1.2, while — 1.5 | ввёл −5, потом 30000 — один раз «Цель должна быть больше нуля» |
| 2 | 7 дней через for, шаги копить в total | for, range, накопитель — 1.5 | после каждого дня строка «Итого: …», дней не больше 7 |
| 3 | −1 — пропустить день; цель выполнена — выйти | if — 1.4, continue и break — 1.5 | −1 даёт «День пропущен», итог не меняется; после «Цель выполнена» вопросов нет |
| 4 | Итог за неделю и имя по буквам | for по строке — 1.5, end — 1.2 | «Отчёт для: М а к с и м» |

Запускай после каждого шага. Ошибка — читай traceback снизу вверх, а не меняй код наугад.

# Заготовка: замени комментарии кодом

steps.py

```
# Шаг 1
# спроси имя и цель на неделю (цель — число)
# пока цель <= 0: «Цель должна быть больше нуля» и спроси снова

# Шаг 2
# накопитель total = 0
# для каждого дня с 1 по 7:
    # спроси, сколько шагов в этот день
    # Шаг 3: если ввели -1 — «День пропущен» и к следующему дню
    # прибавь шаги к total, напечатай «Итого: ...»
    # Шаг 3: если total >= цели — «Цель выполнена» и выйди из цикла

# Шаг 4
# напечатай «За неделю: ... шагов» и имя через пробелы
```

Каждую строку своего кода ты должен уметь объяснить. Отступы в заготовке показывают, что стоит внутри цикла.

# Готово, если...

**while:** ввёл цель –5 — программа пишет «Цель должна быть больше нуля» и спрашивает снова

**for:** после каждого дня печатается «Итого: ...», дней не больше 7

**break и continue:** –1 печатает «День пропущен» и не меняет итог; после «Цель выполнена» дни больше не спрашиваются

**строка:** в конце «Отчёт для: М а к с и м»

### ★ Рекорд и среднее

В конце добавятся строки «Рекорд недели: ...» и «В среднем за день: ...» — пропущенные дни не считаются.

### ★★ Серия

Добавится строка «Самая длинная серия дней от 5000 шагов: ...». Пропуск или день меньше 5000 обрывает серию.

# Тренажёр таблицы умножения

Сделай тренажёр для младшего брата, сестры или для себя перед контрольной.

- Программа спрашивает, на какое число тренируемся
- По очереди задаёт 10 примеров: «7 x 1 = ?», «7 x 2 = ?» …
- Неверный ответ — показывает правильный и считает ошибку
- Три ошибки — «Игра окончена», дальше примеры не задаются
- В конце — сколько правильных ответов и оценка: «Отлично», «Хорошо» или «Повтори таблицу»

Сам реши, сколько правильных ответов нужно для каждой оценки.

# Итоги

**Сегодня я научился:**

- повторять действия, пока условие верно
- считать шаги и накапливать сумму в цикле
- останавливать зависшую программу и находить, почему она зависла
- повторять действие нужное число раз с нужным началом, концом и шагом
- перебирать строку по буквам
- выходить из цикла досрочно и пропускать шаг

**В следующий раз**

Цикл внутри цикла, случайные числа (random), else у цикла и while True.

Сделаем игру «Угадай число».